CRYPTOGRAPHIC STUDY / PUBLIC RESEARCH BRIEF

Verify the bytes.
Test the security claims.

SPARTACO / VPSI v1 | October 12, 2026 | System author: I. Villasmil

SPARTACO — CRYPTOGRAPHIC STUDY — PUBLIC RELEASE

Abstract

We release for independent study a small corpus of cryptographic artifacts produced by **SPARTACO**, a personal integrity-and-confidentiality system built on the VPSI framework. The corpus consists of: (1) a **sealed private signing key** (32-byte Ed25519 seed), (2) its **sealed key kit** (keyset secrets), and (3) a **sealed image document**. The two key artifacts are wrapped in PNG containers carrying an externally Ed25519-signed manifest; the sealed document is a bare AES-256-GCM envelope. Each artifact is published separately on the author's account under its own file name, as listed in Section 1; the present document is the study sheet that describes them, and every claim it makes is reproducible from the published bytes. The release is intended for researchers and practitioners who wish to attempt recovery of the enclosed secrets under the stated assumptions.

Layering. For the two key artifacts: a PNG transport container (CRC-checked chunk `spRt`); an **outer Ed25519-signed manifest** whose canonical body carries `nucleo = SHA-256(blob)`, half-file channel digests `S` and `Q`, byte count, and a per-file **neutral seal** (leading-zero-bit profile across eight content fragments, plus a mod-3 proof-of-work marker `#VPSI-NEUTRO:`n); and an **inner AES-256-GCM envelope** whose key `K_doc` is derived by HKDF-SHA256 from per-account keyset material (IKM) over domain-separated info strings bound to the artifact ID, the plaintext digest, and the authority fingerprint. The full algorithm suite is declared *inside the signed envelope* ("apartado"), so any tampering with the declared configuration invalidates the outer signature. The sealed document uses the inner envelope only.

Two authorities, by design: public verification — anyone holding the public key can confirm outer signatures, channel digests, seal coherence, and envelope digests — and **confidential recovery** — only the account's motor, deriving K_doc from the account's IKM inside an authenticated session, can open.

Status of the released samples: both outer signatures verified correctly under the published public key; all recomputed digests matched; no plaintext secret is observable in the artifacts. **These are design properties and reproduced observations, not proofs of resistance to every attack**: the challenge below defines what remains open. Nothing in this release requires trust in the author's claims — every table is reproducible from the published bytes.

1 Published artifacts (each file posted separately under its own name)

The four artifacts are published individually on the author's account, each under the exact file name below. Anyone reproducing the tables in this study must hash the downloaded file and compare against the SHA-256 reference here; that single check removes any doubt about which bytes the study describes.

2 How the pieces fit (and why the key alone does not open)

The design enforces a strict division of labor. **Signing** belongs to Ed25519: the private seed signs manifests; the public key verifies them. **Sealing** belongs to AES-256-GCM: the content key `K_doc` is not stored anywhere — it is derived, on demand, by the motor via HKDF-SHA256 from the account's IKM with domain-separated info strings binding the derivation to the artifact ID, the plaintext digest, and the authority fingerprint. Consequently, by design:

- Holding the **sealed artifacts** alone is intended to reveal no plaintext: the ciphertext is computationally indistinguishable from noise, and the manifest exposes only digests and the declared suite.
- Holding the **public key** enables verification (outer signature, channels `S/Q`, seal coherence, envelope digest) but no decryption step references the public key.
- Holding the **private signing seed** does not enable decryption by design: Ed25519 and AES-GCM are disjoint authorities in this scheme.

File name (published as-is)	What it is	Size	SHA-256 (reference)
`rol_PRIV.png`	Sealed private signing key (32-byte Ed25519 seed): PNG + signed outer manifest + AES-256-GCM envelope	7,636 B	`aef87d7e5f66a45e fe4fe22488e2e3b57 88d9e0ade5b57e11 3539f14a708dc12`
[2pt] `rol_KIT.png`	Sealed key kit (keyset secrets): same three-layer construction	8,213 B	`6f6b81e9f77be4a2 24167ac54512289 309ccf8f5150b59f f14d1ff81c720e25b`
[2pt] `SC_IMG_9115.jpeg.spartaco`	Sealed image document: AES-256-GCM envelope (confidentiality-only path, no outer manifest)	226,874 B	`ab501106067b55a8 0cba85d6716b841 018c3bf9b355db58 14d0cb33cef5c8f94`
[2pt] `rol_PUB.pub`	Public verification key, raw Ed25519 (plaintext, 32 bytes)	32 B	`0f63a34dce0fba56 1495b5ce16d53738 75d666cf55e77c45 dd494fb7eb291093`

- Opening requires the account's **keyset secrets**, themselves sealed inside the kit, unwrapped by the motor only inside an authenticated session (private key + kit + unlock material, validated against binding records). This is the intended single path to `K_doc`.

3 Observations reproduced on the released bytes

Check (reproduced)	`rol_PRIV.png`	`rol_KIT.png`	`.spartaco`
Outer Ed25519 signature (canonical body, published public key)	**VALID**	**VALID**	**no outer manifest (envelope-only path)**
[2pt] `nucleo = SHA-256(blob)`	**MATCH**	**MATCH**	n/a
[2pt] Half-file channels `S, Q`	**MATCH**	**MATCH**	n/a
[2pt] Neutral seal (leading-zero profile, mod-3 PoW)	**COHERENT**	**COHERENT**	n/a
[2pt] `ciphertext_digest` recomputed over declared ciphertext span	**MATCH**	**MATCH**	**MATCH**
[2pt] Suite declaration ("apartado") inside signed envelope	**PRESENT**	**PRESENT**	**PRESENT**
[2pt] Plaintext secret observable in the artifact	**NONE**	**NONE**	**NONE**

3.1 Reproducing the envelope digest (corrected method)

Each sealed file stores its payload after a literal separator line `--SPARTACO-DOC--` followed by a newline and a two-byte dash continuation (`--\n`) that belongs to the envelope framing; the ciphertext begins immediately after those three bytes and runs for exactly `ciphertext_length` bytes as recorded in the header. The check is: locate the separator, skip it plus the three framing bytes, take exactly `ciphertext_length` bytes, hash with SHA-256, and compare with `ciphertext_digest`. Verified on all three released files. (Rev. 1 of this sheet described the framing incorrectly as "two blank bytes"; thanks to external review for the correction. The header `ciphertext_length` — not visual framing — is the authoritative span.) For the PNG artifacts, header and payload live inside the CRC-checked `spRt` chunk; for the `.spartaco` file, the header occupies the start of the file.

4 Challenge scope and open questions

The release separates three kinds of statements, and invites two classes of study.

- **Reproduced here (independent of the author):** signature validity under the published public key; digest matches over the exact ciphertext span; seal coherence; absence of observable plaintext secrets.

- **Declared by the design (not proven by this release):** that deriving K_doc requires the account IKM; that the motor's session path is the only opening path; that the neutral seal is unpredictable without the file bytes. These are properties of the implementation as built — studying whether they hold under adversarial conditions is part of the challenge.
- **Open to researchers:** (a) forge an outer manifest that verifies under the published public key; (b) transplant any manifest field between the two signed artifacts and pass verification; (c) recover any plaintext secret (private seed, unlock material, keyset secrets, or the sealed image) from the released bytes alone; (d) produce a payload whose recomputed digests match a manifest not signed by the author.

What this release does *not* claim: it does not claim that the underlying primitives (Ed25519, SHA-256, HKDF, AES-256-GCM) are beyond attack, nor that implementation-level flaws are impossible. It claims only that the reproduced observations above hold on the released bytes. The declared suite binds any reproduction to the exact configuration under which the artifacts were sealed.

SPARTACO / VPSI V1 · RELEASED FOR INDEPENDENT CRYPTOGRAPHIC STUDY · EACH ARTIFACT PUBLISHED SEPARATELY UNDER ITS OWN NAME

REV. 2 INCORPORATES EXTERNAL REVIEW · DESCRIPTIVE, NOT A SECURITY GUARANTEE

This document presents four supplied SPARTACO artifacts for independent cryptographic examination. Two PNG files carry externally signed Ed25519 manifests and sealed payloads; a separate `.spartaco` file carries an AES-256-GCM document envelope without an outer signed manifest; the fourth file is a raw Ed25519 public verification key.

The study separates **publicly reproducible integrity checks** from **confidentiality and legitimate recovery**. All reference hashes describe the exact supplied files. These files were provided for examination; this document alone does not prove their public publication status.

01 / Independent research tracks

TRACK A / INTEGRITY

Can a modified or substituted signed package pass verification under the original trusted public key without the private signing key?

TRACK B / CONFIDENTIALITY

Can any protected plaintext be recovered without the legitimate keyset, IKM and authenticated session material?

02 / Cryptographic mechanisms

DECLARED AND OBSERVED COMPONENTS

Ed25519	Canonical manifest signatures on PRIV and KIT.
SHA-256	Whole-file references, signed blob digest, S/Q channels and ciphertext digest.
HKDF-SHA256	Documented domain-separated key derivation.
AES-256-GCM	Declared authenticated-encryption mechanism.
PNG / `spRt`	Transport of a signed manifest and encrypted blob in role packages.

SCOPE NOTE

Valid Ed25519 signatures and matching hashes establish specified integrity relationships. They do not, by themselves, verify the entire account engine, prove exclusive recovery, or demonstrate immunity to implementation attacks.

03 / Exact reference artifacts

Each full-file SHA-256 digest below is a reference for reproducing the measurements. Two printed lines form one continuous 64-character hexadecimal digest.

FILE 01 / SIGNING ROLE

`rol_PRIV.png` 7,636 bytes
Sealed signing material in a PNG container, with an external signed manifest.
SHA-256 (complete file)
`aef87d7e5f66a45efe4fe22488e2e3b5`
`788d9e0ade5b57e113539f14a708dc12`

FILE 02 / KEY KIT

`rol_KIT.png` 8,213 bytes
Sealed keyset material in a PNG container, with an external signed manifest.
SHA-256 (complete file)
`6f6b81e9f77be4a224167ac545122893`
`09ccf8f5150b59ff14d1ff81c720e25b`

FILE 03 / ENCRYPTED DOCUMENT

`SC_IMG_9115.jpeg.spartaco` 226,874 bytes
AES-256-GCM encrypted-document envelope, with no outer Ed25519 manifest.
SHA-256 (complete file)
`ab501106067b55a80cba85d6716b8410`
`18c3bf9b355db5814d0cb33cef5c8f94`

FILE 04 / PUBLIC VERIFICATION KEY

`rol_PUB.pub` 32 bytes
Raw Ed25519 public key used for signature verification.
SHA-256 (complete file)
`0f63a34dce0fba561495b5ce16d53738`
`75d666cf55e77c45dd494fb7eb291093`

DISTRIBUTION INTEGRITY

When these files are shared separately, recipients should compute the whole-file SHA-256 of each downloaded artifact before relying on the verification results in this document.

04 / Independently checked properties

PRIV and KIT are signed role packages. The image document has an envelope-only format. Therefore not every public-key test applies to all three protected files.

VERIFICATION MATRIX / EXACT SUPPLIED BYTES

Check	PRIV	KIT	Document
Ed25519 signature of canonical body	VALID	VALID	N/A
Full-file SHA-256 reference	MATCH	MATCH	MATCH
PNG chunk CRC	MATCH	MATCH	N/A
Signed blob digest (nucleo)	MATCH	MATCH	N/A
SHA-256 half-file channels S/Q	MATCH	MATCH	N/A
Signed blob byte count	MATCH	MATCH	N/A
Ciphertext digest, exact span	MATCH	MATCH	MATCH
Full neutral-seal PoW algorithm	NOT RUN	NOT RUN	N/A
AES-GCM decryption / tag validation	NOT RUN	NOT RUN	NOT RUN

05 / Ciphertext measured lengths and digests

RECOMPUTED SHA-256 / CIPHERTEXT ONLY

PRIV / 2,497 bytes
`d8f088c180c7afeb4ca8ba283ade3920`
`eb50a0f44e2116d08dcbcd3110e63635`

KIT / 3,000 bytes
`52e9540b96a161348cc62c36b66f7cd3`
`2bb8c624e0d7ba875af5f812b75d7b2f`

DOCUMENT / 225,880 bytes
`6eb657d7de5847b8f3bb339be9537c65`
`7aed2016b69962d2ca3d41e4a10ed5ab`

VERIFICATION INTERPRETATION

The public key verifies each role package's canonical signed manifest. The signed blob digest binds that manifest to the corresponding encrypted blob, including its header. The stand-alone `.spartaco` document does not have the same external signed manifest. Its public ciphertext digest is a consistency check, not an author signature.

06 / Reproduction protocol

STEPS FOR INDEPENDENT RESEARCHERS

1. Obtain all four files and compute their full-file SHA-256 digests.
2. For PRIV/KIT, parse PNG chunks, validate CRC values and locate `spRt`.
3. Extract the signed manifest body and its Ed25519 signature.
4. Verify the signature with the trusted raw `rol_PUB.pub` key.
5. Decode the bound blob; compare its size and SHA-256 digest.
6. Recompute SHA-256 for the S/Q halves and compare to the signed fields.
7. Parse envelope headers and read exactly `ciphertext_length` bytes after the `-ESPARTACO-DOC-` line delimiter.
8. Hash ciphertext bytes only, excluding the final `#VPSI-NEUTRO:n` marker.
9. Record exact commands, inputs, verification outputs and any discrepancies.

07 / Open research challenge

TRACK A / MANIFEST INTEGRITY

Submit a reproducible example of a modified or substituted signed package that passes the original public-key verification. Report which bytes changed and which checks accepted them.

TRACK B / CONFIDENTIALITY

Submit a reproducible recovery of protected plaintext or verified unauthorized opening without the legitimate IKM/session material. Headers, digests or ciphertext samples alone do not count as plaintext recovery.

08 / Mathematical basis of the security boundary

FORMAL BARRIER / CONJUNCTION OF CONTRACTS

Let $\mathcal{C}(r)$ be the set of predicates required for an operation r: integrity, authenticated identity, active binding, authorization, lifecycle state, the Ω gate, and the relevant cryptographic domain.
The SPARTACO acceptance rule, of the "fail-safe" (fail-closed) type, is:

$$\mathrm{GRANT}(r) = \bigwedge_{c \in \mathcal{C}(r)} c(r), \qquad (\exists c : \; c(r) = 0) \Rightarrow \mathrm{DENY}(r).$$

Therefore, a request that fails to meet any of the required conditions cannot be executed *within the defined transition relationship.*
Passing the Ω gate alone cannot compensate for a lack of identity, permissions, or key material.

08 / Mathematical basis of the security boundary

FORMAL BARRIER / CONJUNCTION OF CONTRACTS

Let $\mathcal{C}(r)$ be the set of predicates required for an operation r: integrity, authenticated identity, active binding, authorization, lifecycle state, the Ω gate, and the relevant cryptographic domain.
The SPARTACO acceptance rule, of the "fail-safe" (fail-closed) type, is:

$$\mathrm{GRANT}(r) = \bigwedge_{c \in \mathcal{C}(r)} c(r), \qquad (\exists c : \; c(r) = 0) \Rightarrow \mathrm{DENY}(r).$$

Therefore, a request that fails to meet any of the required conditions cannot be executed *within the defined transition relationship.*
Passing the Ω gate alone cannot compensate for a lack of identity, permissions, or key material.

CRYPTOGRAPHIC BINDING / ABSENCE OF ALTERNATIVE AUTHORITY

For signed PRIV/KIT packages, Ed25519 authenticates the canonical manifest; commitments signed with SHA-256 bind the *blob* and the S/Q digests to it.
Modifying the authenticated content while maintaining verification validity requires bypassing these checks.
The standalone `.spartaco` envelope lacks an external Ed25519 signature.
For opening, the contract requires valid authentication, binding, and authorization, in addition to document-domain-specific material (K_{DOC}) derived via HKDF-SHA256.

AES-256-GCM / AUTHENTICATED ENCRYPTION

AES-256-GCM authenticates the ciphertext and its associated data before releasing the plaintext.
The domains K_{AUTH}, K_{DEF}, K_{DOC}, and K_{REC} have independent authorities:
a public signing key does not allow decryption, and a denied opening does not authorize a Z/S/Q defense transition.

TECHNICAL CONCLUSION

Bypassing a failed contract gate contradicts the acceptance rule; forging valid cryptographic evidence is computationally infeasible under the security of the established primitives.

SPARTACO / VPSI v1

INDEPENDENT CRYPTOGRAPHIC STUDY

FORMAL CONTRACT / INTEGRITY / AUTHORIZATION / RECOVERY

SEALED / AUTHENTICATED

This document has been sealed and signed by SPARTACO. Every byte of the canonical content has been authenticated via Ed25519 signature over the manifest and SHA-256 commitments over the payload.

Any alteration to the sealed content will invalidate the signature and be detected upon verification. The integrity of this document is cryptographically bound; modification without the signing key is computationally infeasible.